

Lab 16: Database Design and Queries: Schema design and SQL generation.

Task 1: Database Schema Design

Prompt:

1. Create a database for an online course platform with instructors, students, and courses.
2. Each instructor can teach many courses but each course has one instructor.
3. Students can enroll in many courses, so create a relation table.
4. Use primary keys for each table and foreign keys for relationships.
5. Use proper data types and keep tables simple and normalized.

Code:

```
Connected.
SQL> CREATE TABLE Instructors (instructor_id INT PRIMARY KEY,name VARCHAR(50),country VARCHAR(50));
Table created.
SQL> CREATE TABLE Courses (course_id INT PRIMARY KEY,title VARCHAR(100),instructor_id INT,FOREIGN KEY (instructor_id) REFERENCES Instructors(instructor_id))
;
Table created.
SQL> CREATE TABLE Students(student_id INT PRIMARY KEY,name VARCHAR(50),email VARCHAR(100));
Table created.
SQL> CREATE TABLE Enrollments(enrollment_id INT PRIMARY KEY,student_id INT,course_id INT,FOREIGN KEY (student_id) REFERENCES Students(student_id),FOREIGN KEY (course_id) REFERENCES Courses(course_id));
Table created.
SQL>
```

Output:

```
SQL> CREATE TABLE Enrollments(enrollment_id INT PRIMARY KEY,student_id INT,course_id INT,FOREIGN KEY (student_id) REFERENCES Students(student_id),FOREIGN KEY (course_id) REFERENCES Courses(course_id));
Table created.
SQL> |
```

Explanation:

1. We created 4 tables for instructors, courses, students, and enrollments.
2. Primary keys uniquely identify each record.
3. Foreign keys connect tables correctly.
4. The design avoids duplicate data (normalization).

Task 2: Insert Sample Data

Prompt:

1. Insert sample data into instructors, courses, and students tables.
2. Add at least 3 instructors, 4 courses, and 3 students.

3. Also insert enrollment data to connect students and courses.
4. Use correct values matching table structure.
5. Check data using SELECT queries.

Code:

```
SQL> INSERT ALL
2      INTO Courses VALUES (101, 'Python', 1)
3      INTO Courses VALUES (102, 'Java', 2)
4      INTO Courses VALUES (103, 'DBMS', 2)
5      INTO Courses VALUES (104, 'AI', 3)
6  SELECT * FROM dual;

4 rows created.

SQL> INSERT ALL
2      INTO Students VALUES (1, 'Rahul', 'rahul@gmail.com')
3      INTO Students VALUES (2, 'Anu', 'anu@gmail.com')
4      INTO Students VALUES (3, 'Kiran', 'kiran@gmail.com')
5  SELECT * FROM dual;

3 rows created.

SQL> INSERT ALL
2      INTO Enrollments VALUES (1, 1, 101)
3      INTO Enrollments VALUES (2, 2, 102)
4      INTO Enrollments VALUES (3, 3, 103)
5      INTO Enrollments VALUES (4, 1, 104)
6  SELECT * FROM dual;

4 rows created.

SQL> |
```

Output:

```
SQL> INSERT ALL
2      INTO Courses VALUES (101, 'Python', 1)
3      INTO Courses VALUES (102, 'Java', 2)
4      INTO Courses VALUES (103, 'DBMS', 2)
5      INTO Courses VALUES (104, 'AI', 3)
6  SELECT * FROM dual;

4 rows created.

SQL> INSERT ALL
2      INTO Students VALUES (1, 'Rahul', 'rahul@gmail.com')
3      INTO Students VALUES (2, 'Anu', 'anu@gmail.com')
4      INTO Students VALUES (3, 'Kiran', 'kiran@gmail.com')
5  SELECT * FROM dual;

3 rows created.

SQL> INSERT ALL
2      INTO Enrollments VALUES (1, 1, 101)
3      INTO Enrollments VALUES (2, 2, 102)
4      INTO Enrollments VALUES (3, 3, 103)
5      INTO Enrollments VALUES (4, 1, 104)
6  SELECT * FROM dual;

4 rows created.

SQL> |
```

Explanation:

1. We inserted sample data into all tables.
2. Each table has required number of records.
3. Enrollments connect students and courses.
4. SELECT queries show all inserted data.

Task 3: CRUD Operations

Prompt:

1. Retrieve all courses available in the system.
2. Update the email of a student.
3. Delete one course from the database.
4. Use SELECT queries to verify changes.
5. Ensure no wrong data is affected.

Code:

```
SQL> SELECT * FROM Courses;
COURSE_ID
TITLE
-----
INSTRUCTOR_ID
-----
101
Python      1
102
Java        2
COURSE_ID
TITLE
-----
INSTRUCTOR_ID
-----
103
DBMS        2
104
AI
COURSE_ID
TITLE
-----
INSTRUCTOR_ID
-----
```

```
SQL> UPDATE Students SET email = 'rahul_new@gmail.com' WHERE student_id = 1;
1 row updated.
```

```
SQL> SELECT * FROM Students;
STUDENT_ID NAME
-----
EMAIL
-----
1 Rahul
rahul_new@gmail.com
2 Anu
anu@gmail.com
3 Kiran
kiran@gmail.com
```

Output:

```
SQL> DELETE FROM Enrollments WHERE course_id = 104;
1 row deleted.

SQL> DELETE FROM Courses WHERE course_id = 104;
1 row deleted.

SQL> SELECT * FROM Courses;

  COURSE_ID
-----
  TITLE
-----
INSTRUCTOR_ID
-----
         101
Python          1
         102
Java           2
  COURSE_ID
-----
  TITLE
-----
INSTRUCTOR_ID
-----
         103
DBMS          2
```

Explanation:

1. SELECT shows all courses.
2. UPDATE changes student email correctly.
3. DELETE removes one course only.
- 4.** Verification ensures correct results.

Task 4: Aggregate Queries

Prompt:

1. Count how many students are enrolled in each course.
2. Group data by course for counting.
3. Find courses with more than 1 enrollment.
4. Use GROUP BY and HAVING clauses.
5. Check results carefully.

Code:

```
SQL> SELECT course_id, COUNT(student_id) AS total_students
2 FROM Enrollments
3 GROUP BY course_id;
```

COURSE_ID	TOTAL_STUDENTS
102	1
101	1
103	1

```
SQL>
```

Output:

```
SQL> SELECT course_id, COUNT(student_id) AS total_students
2 FROM Enrollments
3 GROUP BY course_id
4 HAVING COUNT(student_id) > 1;
```

```
no rows selected
```

```
SQL> |
```

Explanation:

1. GROUP BY groups data by course.
2. COUNT counts students in each course.
3. HAVING filters grouped results.
4. Output shows correct summary.

Task 5: Query Optimization

Prompt:

1. Retrieve courses taught by instructors from a specific country.
2. Write a simple query using JOIN.
3. Improve query using efficient join and indexing.
4. Ensure both queries give same result.
5. Compare performance for large data.

Code:

```
SQL> SELECT title
2 FROM Courses, Instructors
3 WHERE Courses.instructor_id = Instructors.instructor_id
4 AND country = 'India';
```

```
TITLE
```

```
-----
Java
DBMS
```

Output:

```
SQL> SELECT c.title
  2  FROM Courses c
  3  INNER JOIN Instructors i
  4  ON c.instructor_id = i.instructor_id
  5  WHERE i.country = 'India';
```

```
TITLE
-----
```

```
Java
DBMS
```

```
SQL> |
```

```
DBMS
```

```
SQL> CREATE INDEX idx_country ON Instructors(country);
```

```
Index created.
```

```
SQL> |
```

Explanation:

1. Both queries return same results.
2. INNER JOIN is clearer and faster.
3. Index improves search speed.
4. Optimized query works better for big data.